

Introduction à la complexité algorithmique

Dr. Zakaria Sawadogo

zakaria.sawado@gmail.com

Introduction à la complexité algorithmique

- 1 Rappels
- 2 Classes de complexité
- 3 Syllabus du cours

Commençons par un exemple ...

Algorithme Echange1

Variables

var1,var2,tmp :Entier

Début

tmp $\leftarrow$ var1

var1 $\leftarrow$ var2

var2 $\leftarrow$ tmp

Fin

Algorithme Echange2

Variables

var1,var2 :Entier

Début

var1 $\leftarrow$ var1+var2

var2 $\leftarrow$ var1-var2

var1 $\leftarrow$ var1-var2

Fin

Commençons par un exemple ...

Quelles différences entre Echange1 et Echange2

- **Echange1** utilise 3 variables (var1, var2 et tmp) tandis que **Echange2** n'utilise que 2 variables (var1, var2). **Donc si une variable est codée sur 4 octets (entier simple) Echange1 utilise 12 octets de mémoire et Echange2 en utilise 8 octets.**
- Comme opérations, **Echange1** utilise 3 affectations tandis que **Echange2** utilise 3 affectations et 3 opérations arithmétiques. **Donc si une affectation se fait en t_1 temps processeur et une opération arithmétique en t_2 temps processeur Echange1 utilise $3t_1$ temps et Echange2 utilise $3t_1 + 3t_2$ temps.**

Commençons par un exemple ...

Problématique

Imaginez maintenant qu'on veuille échanger les valeurs des cases de 2 tableaux de 1000, 10^6 , 10^9 , ... éléments. **Quelles sont les contraintes liées à l'utilisation de Echange1 et Echange2 ?**

- 1 Rappels
- 2 Classes de complexité
- 3 Syllabus du cours

Rappels

Algorithme

Un algorithme est une description d'une méthode de résolution d'un problème.

Structure d'un algorithme ?

Un algorithme a généralement :

- Une entête pouvant contenir :
 - le nom de l'algorithme ;
 - la déclaration des variables en entrée et en sortie ;
 - la déclaration de types de données particuliers ;
 - la déclaration de fonctions ou procédures.
- Un corps contenant un ensemble d'instructions destinées à résoudre le problème.

Rappels

Contenu d'un algorithme

Un algorithme contient des données et des traitements :

- Les données sont organisées en :
 - constantes ;
 - variables ;
 - tableaux ;
 - listes (graphes, arbres).
- Les traitements sont aussi organisés en :
 - séquences ;
 - structures conditionnelles ;
 - boucles.

Rappels

Complexité d'un algorithme

La complexité d'un algorithme est une mesure de son efficacité. Elle peut se définir de deux façons :

- **complexité spatiale** : c'est la taille des données générées par l'algorithme ;
- **complexité temporelle** : c'est le nombre d'opérations liées à l'exécution de l'algorithme.

Rappels

Évaluation de la complexité d'un algorithme

- De façon **théorique** : au niveau de l'algorithme par l'analyse et le calcul ;
- De façon **expérimental** : au niveau de l'exécution du programme.

Rappels

Évaluation expérimentale

- L'évaluation expérimentale est très tributaire des langages de programmation, des systèmes d'exploitation et des ordinateurs ;
- Elle requiert l'utilisation d'un logiciel de **benchmark** (étalonnage) qui donne une mesure de puissance d'un ordinateur en flops (floating point operations per second) en fonction des traitements effectués (calculs bruts, sur des entiers ou des réels, calculs liés à l'affichage, ...)

Rappels

Évaluation théorique

- On s'intéresse au nombre d'opérations en fonction de la taille et la nature des données ;
- Dans la suite on notera :
 - **n** la taille des données ;
 - **T(n)** le nombre d'opérations élémentaires.
- Dans l'évaluation du nombre d'opérations on va considérer :
 - des opérations comme l'affectation d'entiers : **ae** ;
 - des opérations comme la comparaison d'entiers : **ce** ;
 - des opérations élémentaires comme les calculs élémentaires d'entiers : **oe** ;
- On va négliger les opérations de déclarations et de retour.

Rapports

Évaluation théorique : terminologies

- **Complexité au pire des cas** : le nombre d'opérations maximal ;
- **Complexité au meilleur des cas** : le nombre d'opérations minimal.

Complexité asymptotique

Dans la pratique, on n'évalue pas exactement le nombre d'opérations mais :

- soit la **complexité au pire des cas** pour borner ;
- soit la **forme générale** de la complexité pour avoir l'allure ;
- ou le **comportement asymptotique** par rapport à un groupe de complexité connu.

Rappels

Domination asymptotique $f = O(g)$

Pour f et g , deux fonctions on dit que f est dominée asymptotiquement par g et on note $f = O(g)$ s'il existe deux constantes $c > 0$ et n_0 telles que pour $x > n_0$ on a $f(x) < c * g(x)$.

Propriétés de la notation de Landau

La notation O dite **rond de** est la notation de Landau. Elle a les propriétés de :

- **transitivité** : $f = O(g)$ et $g = O(h)$ impliquent $f = O(h)$
- **combinaison linéaire** :
 - $f = O(g)$ et K un entier non nul alors $K * f = O(g)$
 - $f_1 = O(g_1)$ et $f_2 = O(g_2)$ impliquent $f_1 + f_2 = O(g_1 + g_2)$ et $f_1 * f_2 = O(g_1 * g_2)$

Rappels

Domination asymptotique $f = O(g)$

Pour f et g , deux fonctions on dit que f est dominée asymptotiquement par g et on note $f = O(g)$ s'il existe deux constantes $c > 0$ et n_0 telles que pour $x > n_0$ on a $f(x) < c * g(x)$.

Propriétés de la notation de Landau

La notation O dite **rond de** est la notation de Landau. Elle a les propriétés de :

- **transitivité** : $f = O(g)$ et $g = O(h)$ impliquent $f = O(h)$
- **combinaison linéaire** :
 - $f = O(g)$ et K un entier non nul alors $K * f = O(g)$
 - $f_1 = O(g_1)$ et $f_2 = O(g_2)$ impliquent $f_1 + f_2 = O(g_1 + g_2)$ et $f_1 * f_2 = O(g_1 * g_2)$

Introduction à la complexité algorithmique

- 1 Rappels
- 2 Classes de complexité**
- 3 Syllabus du cours

Classes de complexité

Définitions

On peut classer les algorithmes en fonction de leur complexité : on parle de classes de complexité. On distingue :

- complexité constante ;
- complexité logarithmique ;
- complexité linéaire ;
- complexité quasi-linéaire ;
- complexité quadratique ;
- complexité polynomiale ;
- complexité exponentielle ;
- complexité factorielle.

Classes de complexité

Définitions

On peut classer les algorithmes en fonction de leur complexité : on parle de classes de complexité. On distingue :

- complexité constante ;
- complexité logarithmique ;
- complexité linéaire ;
- complexité quasi-linéaire ;
- complexité quadratique ;
- complexité polynomiale ;
- complexité exponentielle ;
- complexité factorielle.

Classes de complexité

Complexité constante

- Le temps d'exécution n'augmente pas quand la taille des données augmente ;
- On note $O(1)$.

Exemple

Écrire une fonction de permutation des valeurs des éléments i et j d'un tableau T :

- avec utilisation d'une variable temporaire ;
- sans utilisation d'une variable temporaire.

Classes de complexité

Complexité linéaire

- L'augmentation du temps d'exécution est proportionnelle à l'augmentation de la taille des données ;
- On note $O(n)$.

Exemple

Écrire une fonction de calcul de la somme des valeurs d'un tableau T de taille n .

Classes de complexité

Complexité linéaire

- L'augmentation du temps d'exécution est proportionnelle à l'augmentation de la taille des données ;
- On note $O(n)$.

Exemple

Écrire une fonction de calcul de la somme des valeurs d'un tableau T de taille n .

Classes de complexité

Complexité quadratique

- L'augmentation du temps d'exécution est multiplié par 4 à l'augmentation de la taille des données par 2 ;
- On note $O(n^2)$.

Exemple

Écrire une fonction qui ordonne un tableau T de taille n .

Classes de complexité

Complexité quadratique

- L'augmentation du temps d'exécution est multiplié par 4 à l'augmentation de la taille des données par 2 ;
- On note $O(n^2)$.

Exemple

Écrire une fonction qui ordonne un tableau T de taille n .

Classes de complexité

Complexité polynomiale

- L'augmentation du temps d'exécution est multiplié par 2^i à l'augmentation de la taille des données par 2. C'est le cas des algorithmes qui contiennent i boucles imbriquées ;
- On note $O(n^i)$.

Exemple

Écrire une fonction d'affichage des combinaisons de i chevaux parmi n .

Classes de complexité

Complexité factorielle

On note $O(n!)$ et ça correspond à la classe des algorithmes asymptotiquement équivalents à $O(n^n)$.

Exemple

Écrire une fonction de résolution du problème du voyageur de commerce.

Fonction d'Ackermann

C'est une des pires complexités qu'on puisse avoir :

- $\text{Ack}(m,n)=n+1$ si $m=0$;
- $\text{Ack}(m,n)=\text{Ack}(m-1,1)$ si $n=0$ et $m>0$;
- $\text{Ack}(m,n)=\text{Ack}(m-1,\text{Ack}(m,n-1))$ sinon.

Classes de complexité

Exercice (TD/TP)

- Écrire l'algorithme correspondant à la fonction d'Ackermann ;
- Implémenter l'algorithme en C ;
- Tester pour $m=3$ et $n=2$ et commenter l'exécution et le résultat.

Introduction à la complexité algorithmique

- 1 Rappels
- 2 Classes de complexité
- 3 Syllabus du cours**

Objectif du cours

Objectif général

Maîtriser les fondamentaux de la calculabilité, de la décidabilité et de la complexité algorithmique.

Objectifs spécifiques

- Déterminer la classe de complexité d'un algorithme ;
- Déterminer et comparer les temps d'exécutions d'algorithmes ;
- Déterminer si un problème est NP et déterminer une approche pour le résoudre.

Objectif du cours

Objectif général

Maîtriser les fondamentaux de la calculabilité, de la décidabilité et de la complexité algorithmique.

Objectifs spécifiques

- Déterminer la classe de complexité d'un algorithme ;
- Déterminer et comparer les temps d'exécutions d'algorithmes ;
- Déterminer si un problème est NP et déterminer une approche pour le résoudre.

Plan du cours de complexité algorithmique

Plan

- 1 **Calcul de complexité** : cours théorique et TD ;
- 2 **Calcul de temps d'exécution** : cours théorique et TP ;
- 3 **Problème NP et algorithmique avancée** : cours théoriques et travaux de groupes.

Évaluation

- 1 **Rapport de TP** : calcul de temps d'exécution ;
- 2 **Examen sur table.**